1. Executive Summary & Objective

In this mini project, I designed, trained, evaluated, and deployed a cloud-native command parsing service. The objective was to take a large, capable teacher language model (Microsoft Phi-3.5-mini-instruct, 3.8B parameters) and distill its command-to-JSON reasoning into an ultra-compact student model (EleutherAI Pythia-70M, 70M parameters), deploy it as a containerized microservice on a local Kubernetes cluster (k3d), and evaluate it under realistic cloud operational conditions with strict resource limits.

2. What I Learnt

Through this end-to-end project, I gained deep practical understanding across several core cloud computing and NLP engineering

concepts:
- Sequence-Level Knowledge Distillation: I learnt how smaller language models can be supervised by treating teacher-generated outputs as target sequences. By masking input prompt tokens with label -100, the student computes cross-entropy loss strictly over target JSON tokens, effectively acting as an automated sequence translator.
- Cloud Decoupling & Object Storage: I learnt how cloud applications decouple stateless compute from stateful storage. By provisioning SeaweedFS in the data namespace and streaming model weights on pod startup via S3 APIs, container images remained lightweight (1.66 GB) without embedding heavy weights into Docker layers.
- Stateless vs Stateful Network

Security: I gained practical experience designing Network Access Control Lists (NACLs). Because stateless filters evaluate each packet in isolation without session tracking, outbound replies to clients require explicit ephemeral port allow rules (ports 1024-65535).
- Kubernetes Orchestration & Health Probes: I understood the vital distinction between liveness (/healthz) and readiness (/readyz) probes. The readiness probe successfully prevented Kubernetes from routing user requests to pods while model weights were still downloading from S3 into memory.

3. Architectural Discoveries on Model Architectures
- Pythia-70M Suitability: A 70-million-parameter causal language model is small enough to train and run

CPU inference on a commodity laptop with under 1 GiB RAM. However, its compressed representation makes it vulnerable to syntax degradation and repetition loops without careful temperature and max-token limits.

- Teacher Parallelization:
Generating labels with Phi-3.5 locally via Ollama across 3,000 commands was computationally intensive. Modulo sharding across worker pods (line index % SHARD COUNT) demonstrated linear scaling of batch inference workloads across cloud worker pools.

4. Engineering Challenges Faced & Solutions Implemented

- Challenge 1: Teacher Model Hallucinations and Inconsistent JSON Outputs. Despite setting temperature=0 and seed=42,

Phi-3.5 occasionally emitted invalid JSON syntax, markdown fences, illegal intent categories, and hallucinated slot values not present in the user's spoken command. To resolve this, I built a strict, five-stage validation pipeline in worker/clean.py that enforced assignment rules (a) through (e). The cleaner verified exact JSON syntax, verified the four required keys (intent, date, time, place), verified the closed set of 8 intents, and checked verbatim slot substrings in the command text. It discarded 708 invalid/duplicate records, yielding a clean dataset of 2,292 records.

- Challenge 2: Infeasibility of Logit-Based Distillation. I explored whether logit-based distillation could transfer richer probability distributions than sequence-level distillation. My analysis proved that

logit-based distillation was technically impossible in this setup for two reasons: First, Ollama only exposes a text generation endpoint (/api/generate) and does not expose vocabulary logits or forward activations. Second, Phi-3.5 uses a 32,064-token SentencePiece tokenizer, whereas Pythia-70M uses a 50,304-token GPT-NeoX BPE tokenizer. Without an aligned projection matrix, token-level KL divergence cannot be computed. Therefore, sequence-level distillation was the optimal, viable approach.

- Challenge 3: Stateless NACL Asymmetric Traffic Dropping. Initial public subnet NACL configurations dropped legitimate HTTP/S responses returning to external clients, as well as backend responses returning from the app subnet. I diagnosed that stateless NACLs do not track

connection handshakes. When an external client sends an HTTP request, the response packet travels outbound to a dynamically chosen client ephemeral port (1024-65535). By adding Rule 110 outbound (0.0.0.0/0, ports 1024-65535) and Rule 140 inbound (10.20.0.0/22, ports 1024-65535), packet flow was fully restored, achieving 100% (6/6) in tools/ndcl_sim.py.

- Challenge 4: Strict Container Resource Constraints (1 CPU, 1 GiB RAM). PyTorch applications easily exceed 1 GiB memory and crash with OOMKilled when running concurrent inferences. In service/app.py, I implemented torch.inference_mode(), loaded weights with ~~float~~float32 CPU tensors, and guarded inference with a threadd mutex. Sequential testing with 50 requests under hard

container limits (1 CPU, 1 GiB) finished with 0 OOM kills and a measured peak memory of 737.4 MiB.

5. Empirical Findings & Benchmarking Results

- Model Quality Comparison (on 100 Gold Validation Examples):
JSON validity reached 90.0% for the distilled student, compared to 94.0% for the teacher and 0.0% for the untrained baseline. Intent accuracy was 75.0% for the student versus 85.0% for the teacher. Full-frame accuracy was 41.0% for the student compared to 44.0% for the teacher. The distilled 70M model retained 93.2% of the teacher's full-frame accuracy despite being 54 times smaller (70M vs 3.8B parameters).
- Locust Load Test Measurements (4 Concurrent Users, 2 Minutes):

The service processed 89 total requests with 78 successful responses and 11 failures (12.36% error rate). Overall throughput was 0.748 requests/second, median latency (p50) was 3,700 ms, and 95th percentile latency (p95) was 16,000 ms. Pod 1 served 41 requests (52.6%) and Pod 2 served 37 requests (47.4%), confirming that both replicas served well over 30% of total traffic. Three cold-start pod eviction trials yielded 43.8s, 41.2s, and 42.5s, giving a median cold-start latency of 42.5 seconds.

- Hypothetical Cloud Cost Modeling: Using the rate card, each replica costs 3.50 INR for 1 vCPU and 0.45 INR for 1 GiB RAM, totaling 3.95 INR per hour. Two running replicas cost 7.90 INR per hour. At the measured throughput of

2,340 requests per hour (78 requests per 120 seconds), the cost per 1,000 requests was calculated as (7.90 / 2340) * 1000 = 3.376 INR.

- Service Level Objective (SLO) Accounting: The defined SLO target was 99.0% request success over a 2-minute load-test window. For 89 requests, the allowable error budget was 0 failures. Because 11 failures occurred due to CPU-bound queue saturation on the laptop hardware, 1,236% of the error budget was consumed, resulting in an SLO breach.

6. Critical Limitations & Proposed Engineering Improvements

- Model Syntax Errors: Although JSON validity reached 90%, 10% of outputs failed schema checks. As an improvement, implementing constrained grammar decoding

(using frameworks such as Outlines or Guidance) would mathematically restrict token transitions to valid JSON tokens only, guaranteeing 100% syntax validity.

- CPU Inference Latency: Unquantized autoregressive decoding on CPU caused high p95 latency. Quantizing model weights to INT8 or INT4 using ONNX Runtime or GGML/llama.cpp would yield an estimated 3x to 4x latency reduction.

- Storage Resilience: SeaweedFS ran on a single volume with no high-availability replication. Deploying a multi-AZ MinIO or Ceph cluster with S3 bucket replication factor of 3 would eliminate single points of failure.

- Transport Security: Cluster Ingress handled unencrypted HTTP traffic. Integrating cert-manager